Centro Regional Universitario Córdoba IUA

Ing. en Informática

SISTEMAS EN TIEMPO REAL

Mag. Ing. José Ducloux
jducloux@iua.edu.ar

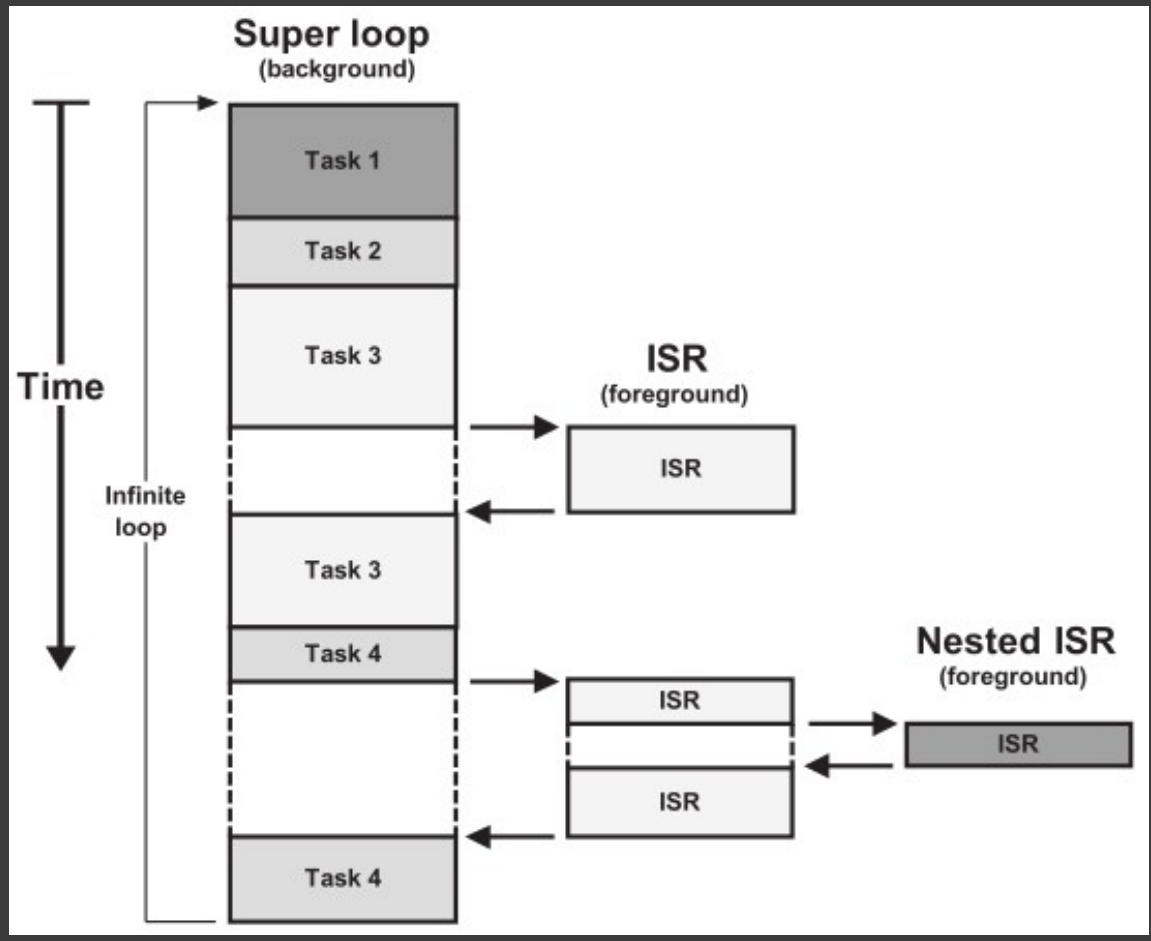
2026

Implementación de software de tiempo real en C

- **Ejecutivo cíclico.**
- POSIX de tiempo real: Interfaz Portable de Sistema Operativo de tiempo real.
- RTOS: Sistema Operativo de Tiempo Real.

Ejecutivo cíclico

- Conocido también como foreground/background, o super lazo.
- Se utiliza en sistemas de baja complejidad.
- Una aplicación consiste de un lazo infinito que llama a los módulos o tareas para realizar las operaciones deseadas.
- Las rutinas de servicio de interrupción controlan los eventos asíncronos.
- Primer plano o foreground se denomina al nivel de interrupción.
- Segundo plano o background se denomina al nivel de tarea.



Ejecutivo cíclico

- Las tareas se ejecutan secuencialmente por turnos.
- Las tareas deben hacer el trabajo rápidamente.
- Las tareas generalmente almacenan sus estados en variables.

```
1  void main (void)
2  {
3      for ( ; ; ) {
4          task0 ( ) ;
5          task1 ( ) ;
6          task2 ( ) ;
7          /* and so on . . . */
8      }
9  }
```

Ejecutivo cíclico

- En su forma más simple:
  - No hay temporizadores (*timers*).
  - No hay mecanismos de comunicación entre tareas.
  - No hay prioridades.
  - No hay colas de mensajes.
  - No hay mecanismos de suspensión.

Ejecutivo cíclico

- Se agrega un *timer* para el manejo del tiempo
  - La ISR del timer incrementa un contador global.
  - Las tareas usan este contador para implementar la temporización.

```
1  void timerIsr (void)
2  {
3      Tics ++;
4  }
```

Ejecutivo cíclico

```
1  void readAnalogDataTask (void)
2  {
3      static int state=0, timer;
4      switch (state) {
5          case 0:
6              timer = Tics+10;
7              state=1;
8              break;
9          case 1:
10             if (Tics==timer){
11                 readAndProcessAnalogData();
12                 timer=Tics+10;
13             }
14             break;
15     }
16 }
```

- Ejemplo de tarea
  - Interrupción del timer cada 10 ms.
  - La tarea lee un canal AD cada 100 ms.
  - Implementa su propio timer (timer = Tics+10;) para realizar la espera.
  - El tiempo de recorrido por todas las tareas no debe exceder 10 ms.
  - La variable de estado (state) es necesaria para inicializar la tarea.

Ejecutivo cíclico

- Comunicación entre tareas
  - Se realiza a través de un dato global
    - Una señal es simplemente una bandera global.
    - Un mensaje con un dato es una estructura de datos global y una bandera que señala a la tarea receptora que el dato está disponible.
  - Ejemplo
    - La tarea receptora debe comprobar la existencia de cualquier dato que ésta pueda recibir y borrar la bandera después que el dato fue leído.

```
1  void displayDataTask (void)
2  {
3      if (DisplayData.flag == 1){
4          displayOnScreen (&DisplayData);
5          DisplayData.flag = 0;
6      }
7  }
```

Ejecutivo cíclico

- Prioridades
  - No existen prioridades en su forma básica.
  - Las tareas se llaman por turnos en el lazo infinito.
  - Porciones de código de alta prioridad se manejan dentro de la ISR.

- Colas de mensajes
  - No hay colas de mensajes.

Ejemplo

- Un sistema compuesto de 3 llaves y 3 motores. Las llaves comandan a los motores mediante dos posiciones: encendido y apagado. Las llaves deben ser examinadas 10 veces por segundo, y los motores deben ponerse en marcha o ser apagados según sea lo especificado.

```
1  void main (void)
2  {
3      int i;
4      for ( ; ; ) {
5          for (i=0; i<3; i++ )
6              if (switchChanged(i)) changeMotor(i);
7      }
8  }
```

- No hay medición del tiempo. Igualmente, el programa se examinará más de 10 veces por segundo sin causar ningún daño.

Ejemplo

- Ahora se solicita que las llaves deben examinarse en intervalos precisos de 10 veces por segundo.

```
for ( ; ; ) {
    if(OneTenthSecondIsUp) {
        for (i=0; i<3; i++ )
            if (switchChanged(i)) changeMotor(i);
    }
    OneTenthSecondIsUp = 0;
}
```

- La variable global *OneTenthSecondIsUp* se pone a 1 en la rutina de servicio de la interrupción del timer cada 100 milisegundos.

Ejemplo

- Se requiere también la lectura de un sensor de presión que debe leerse cada 50 ms. Se debe abrir una válvula si la presión es mayor que 100 psi. Una vez abierta, la válvula debe cerrarse luego que la presión decaiga por debajo de 90 psi.
  - La variable global *FiftyMsIsUp* se pone a 1 en la rutina de servicio de la interrupción del timer cada 50 milisegundos.

```
if (FiftyMsIsUp) {
    switch (valveState) {
        case CLOSED:
            if (pressure( ) > 100) {
                openValve( );
                valveState = OPEN;
            }
            break;
        case OPEN:
            if (pressure( ) < 90) {
                closeValve( );
                valveState = CLOSED;
            }
    }
    FiftyMsIsUp = 0;
}
```

Ejemplo

- El lazo for() debe atender a ambas tareas

```
for ( ; ; ) {
    checkMotorsSwitches ();
    checkPressure ();
}
```

  - Se pueden agregar más tareas al lazo codificadas como funciones en C.
  - Mientras más funciones o tareas se tengan, mayor será el tiempo de recorrido circular.

Ejemplo

- Se agrega un tarea para manejar datos entrantes ya que el sistema está conectado a una red.

```
1  for ( ; ; ) {
2      checkMotorsSwitches ();
3      checkPressure ();
4      checkDatagrams ();
5  }
```

- Si *checkDatagrams()* no se llama lo suficientemente rápido, algunos datos pueden perderse. Esto se podría solucionar agregando una cola dentro de la ISR de recepción de datos.

¿Cuándo usar ejecutivo cíclico?

- El ejecutivo cíclico suele ser compacto y fácil de usar y de entender.
- Por otro lado, no existen prioridades, se emplean esperas activas y se confiere más responsabilidad al desarrollador.

¿Cuándo usar ejecutivo cíclico?

- El ejemplo anterior tiene tres requisitos principales:
  1. Concurrencia (tres tareas independientes a ser ejecutadas).
  2. Medida de la duración del tiempo (muestreo de llaves y de la presión).
  3. Cola de mensajes (cola de datos entrantes).
- Estas tres características son las que típicamente ofrece un sistema operativo de tiempo real.

Sistema operativo de tiempo real

- El ejemplo anterior se podría escribir como en un sistema operativo de tiempo real como tareas independientes.
- Los servicios del sistema operativo son *pause()*, *waitMsg()*, y *freeMsg()*.
- Las tareas se despiertan periódicamente y se examinan las condiciones.
- Se elimina la espera activa.
- El manejo del timer no depende del desarrollador.
- No se requieren variables de estado.
- Las colas de mensajes son manejadas por el sistema operativo.
- Se manejan prioridades.

```
1  void checkMotorsSwitches (void) {
2      while (TRUE) {
3          pause(100);
4          for (i=0; i<3; i++)
5              if (switchChanged(i))
6                  changeMotor (i);
7      }
8  }
9
10 void checkDatagrams (void) {
11     typeMsg *msg;
12     while (TRUE) {
13         msg=waitMsg(DATA_GRAM);
14         processDataGram(msg);
15         FreeMsg(msg);
16     }
17 }
18
19 void  checkPressure (void) {
20     while (TRUE) {
21         pause(50);
22         if (pressure() > 100) {
23             openvalve ();
24             while (TRUE) {
25                 pause(50);
26                 if (pressure() < 90){
27                     closeValve();
28                     break;
29                 }
30             }
31         }
32     }
33 }
```

Centro Regional Universitario Córdoba IUA

Ing. en Informática

¿Preguntas?